

Auditable by Design: Secure AI-Assisted Migration to Post-Quantum Cryptography

Sergey Mkrtchyan

Abstract—An artificial intelligence (AI) recommendation for post-quantum migration neither authorizes a configuration change nor shows that it occurred, and explaining a decision based on a confidential rule discloses it. Auditable by Design is a transaction contract for cryptographic-configuration changes. A policy gate, re-evaluated before execution, checks each AI proposal against a signed observation; a one-time approval of the exact action permits a compare-and-set change, observed separately and recorded as post-quantum-signed, hash-chained, witnessed evidence. A streaming verifier enforces thirteen cross-statement obligations that reject correctly signed but inconsistent evidence from valid key holders, and an optional relation binds a confidential policy predicate to the same action. The single advisor model evaluated, Claude Sonnet 5 through Claude Code, followed 0/120 planted policy-violating instructions under a hardened prompt and 0/120 under a naive one; under complete proposer compromise, 0/60 such proposals executed. Within-policy manipulation, which the gate permits, executed in 6/48 hardened, 46/48 naive, and 24/24 compromised trials. A RISC Zero succinct proof of the relation takes 67.9 s to generate and 11.7 ms to verify, with 98.0 bits of soundness under the vendor’s toy-model conjecture and an unproven zero-knowledge property. Outside the contract, a four-client panel detects two edits of a Transport Layer Security 1.3 service, each silently downgrading one hybrid client class to classical key exchange; no single class detects both. On one host, a governed transaction takes a steady-state median of 6.81 ms (70% in durable commits), and a 100,000-transaction archive verifies in 62.7 s within 115 MiB.

Index Terms—Artificial intelligence, audit trails, cryptographic agility, large language models, migration, post-quantum cryptography, security assurance, zero-knowledge proofs.

I. INTRODUCTION

Post-quantum cryptography (PQC) is becoming an obligation in government and regulated environments. In the United States, Executive Order 14412 directs the Director of the Office of Management and Budget (OMB) to issue guidance requiring agencies to transition federal high-value assets and high-impact systems to PQC key establishment by December 31, 2030, and to PQC signatures by December 31, 2031. It also orders a proposed rule that would require covered contractors to comply with the applicable National Institute of Standards and Technology (NIST) Federal Information Processing Standards (FIPS) by December 31, 2030 [1]. OMB Memorandum M-26-15 implements the federal program [2]. The National Security Agency (NSA) states that Committee on National Security Systems Policy 15 (CNSSP 15) requires

new commercial national security systems to support the Commercial National Security Algorithm Suite 2.0 (CNSA 2.0) by 2027, with legacy systems phased out by 2030 [3]. The European Commission targets the end of 2030 for protecting critical infrastructure under a coordinated roadmap for Member States [4], [5], and the United Kingdom’s National Cyber Security Centre recommends completing the highest-priority activities by 2031 and migration by 2035 [6]. Organizations must turn applicable deadlines into executable changes and retain evidence of what changed, under whose authority, and with what outcome.

In FIPS 203 and 204, NIST standardized the Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM) and the Module-Lattice-Based Digital Signature Algorithm (ML-DSA) [7], [8]. A migration also changes protocols, certificates, hardware security modules, dependencies, and recovery procedures [9], [10]. Artificial intelligence (AI) can assist with discovery, dependency analysis, and configuration proposals, but its recommendations can rest on stale inventories, and large language model (LLM) agents given ordinary engineering prose have produced validated post-quantum downgrades [11].

A recommendation establishes neither permission to change a system nor evidence that the change occurred, and such evidence must remain checkable against holders of valid signing keys. Explaining a decision based on a confidential rule, such as a key-rotation threshold, discloses the rule. Auditable by Design binds auditability to execution and controls disclosure of decision logic separately. Prior work provides hash-chained, witnessed, and post-quantum-signed logs [12]–[16]; the separation of proposal from authority, exact-action approval, commit-time revalidation, and per-decision evidence [17]–[20]; and zero-knowledge proof systems [21], [22]. This paper connects them at the migration boundary and measures where the guarantee ends:

- 1) *A transaction contract for cryptographic-configuration changes* (Section IV). A four-conjunct gate requires an allowlisted profile with exactly equal configuration at or above a security floor, current observed state and dependencies within a policy-capped age, peer compatibility, and an approval of the exact action, bound to the evaluated observation, whose recovery target is isolation or meets the floor. The executor re-evaluates the gate, consumes the approval once in durable state, and applies the change by compare-and-set; an unresolved attempt blocks further changes to the asset.
- 2) *Verification obligations that hold against key holders* (Section V-D). Thirteen obligations relate each transaction’s signed statements to one another, to the cited

Manuscript prepared October 2026; regulatory references reflect the policy context available on October 1, 2026.

S. Mkrtchyan is an independent researcher in Yerevan, Armenia (e-mail: blacksparrow@hotmail.com; ORCID: 0009-0007-8442-1409).

policy, and to earlier records. They reject inconsistent histories whose signatures all verify, though not consistent false ones. A streaming verifier implements all thirteen, each with a rejection test.

- 3) *A confidential-predicate relation bound to the authorized action* (Section VI). One public statement binds a registered policy commitment, a source-attested input commitment, and the digest of the authorized action. A RISC Zero guest program named in the signed policy registration proves it (Section VIII-G); its soundness is conjectured (43.5 bits proven), and zero knowledge is assumed, since the vendor has not proven it [23].
- 4) *The containment boundary with an LLM advisor* (Section VIII-F). Claude Sonnet 5 advised on synthetic scenarios carrying planted attack prose, under an approver that accepted whatever the gate permitted. Under complete proposer compromise, 0/60 policy-violating and 24/24 within-policy proposals executed; the latter marks the limit of enforcement. The model followed 0/120 (hardened prompt) and 0/120 (naive prompt) policy-violating attacks; within-policy manipulation executed in 6/48 and 46/48 trials, respectively, and under the hardened prompt only through free text inside the signed snapshot.
- 5) *Wire-level observation of a deployed service* (Section VIII-H). Outside the contract, a panel of four reference clients reads the key-exchange group that a Transport Layer Security (TLS) 1.3 service negotiates with each. Two group-list edits each validate and keep serving but silently move a different hybrid client class to a classical group; only the panel detects both, consistent with Han’s single-peer argument [11].

II. BACKGROUND AND RELATED WORK

A. PQC Migration and Cryptographic Agility

Näther et al. find inconsistent migration terminology, phases, and roles [24]; Meta’s enterprise account covers inventory, dependencies, implementation, and guardrails [25]. NIST Cybersecurity White Paper (CSWP) 39 describes crypto-agility practices [10], and draft CSWP 48 maps PQC migration capabilities to the NIST Cybersecurity Framework 2.0 and to NIST Special Publication (SP) 800-53 controls [26]. These sources state what an organization should control; this paper specifies how a single configuration change is permitted, executed, and evidenced.

B. AI-Assisted Migration and Runtime Governance

Kim et al. automate a staged hybrid PQC-TLS migration in a policy-as-code pipeline; in 43.6% of trials with a response, errors in LLM-generated migration code escaped static validation and surfaced only at runtime [27]. Han shows that LLM agents editing server configurations from ordinary engineering prose produced validated post-quantum downgrades in 40 of 40 episodes, and argues that no single peer can soundly gate post-quantum delivery [11]. Both results support keeping AI output from deciding or recording a change. Section VIII-H

reproduces the silent downgrade and detects it with a panel of reference clients.

Mazzocchi’s Aegis enforces cryptographic runtime policy for autonomous AI systems [28] and mediates model action proposals through trusted policy decisions and fail-closed execution [17]; Beshane binds policy decisions to the exact policy source with durable signed receipts [18]. AID-Guard revalidates an exactly approved request and the provider state at commit time and yields at most one provider effect across retry and recovery; it is evaluated against live providers and under complete proposer compromise [19]. AIREP separates decision, control, execution, and effect evidence, anchors chain heads to witnesses, and states that its assurance classes do not establish event truth [20].

Table I lists these mechanisms as prior work; AID-Guard evaluates effect paths more broadly. This paper specializes them to cryptographic-configuration transitions: allowlisted profiles matched exactly, freshness bound to the observed peer capabilities and implementations, and recovery restricted to service isolation or a profile at or above a security floor (Section IV).

C. Tamper-Evident and Post-Quantum Evidence

Schneier and Kelsey introduced hash-chained logs that keep entries written before a compromise tamper-evident [12]; Crosby and Wallach add membership and consistency proofs [13]; Certificate Transparency specifies publicly auditable append-only logs [14]. The in-toto framework verifies role-signed supply-chain steps against a layout and so rejects validly signed but inconsistent chains [29]. Kao formalizes post-quantum audit evidence [15], and an Internet-Draft profiles ML-DSA-65-signed, canonically encoded, hash-chained receipts for AI-agent actions [16]. Section V follows this practice. The obligations of Section V-D apply the layout principle of in-toto to authorization semantics, against an adversary holding valid role keys; as with AIREP, an internally consistent false history remains possible until the compromise is contained.

D. Explanation, Evidence, and Confidentiality

Feature-attribution methods explain model predictions [30], [31], but an attribution neither authorizes a change nor observes the deployed configuration. Explanations can also leak protected information [32]; in migration the leak is direct when the requested explanation is itself a protected policy parameter, which no more faithful explanation avoids. Kroll et al. propose commitments and zero-knowledge proofs to hold decisions under a secret policy accountable [33]. Gopalakrishna implements zero-knowledge predicate proofs between AI agents with Bulletproofs and a public threshold, and tests enclave-attested source binding only against a mock authority [34]. The relation of Section VI hides the policy parameters and the measured input behind commitments, binds the public statement to the action digest also bound by the authorization and the receipt, and uses hash-based succinct receipts instead of discrete-logarithm-based Bulletproofs; its zero-knowledge property is an assumption.

TABLE I
RELATIONSHIP TO SELECTED PRIOR WORK

Work	Established there	Added here
PQC guidance and migration [10], [24]–[26]	Agility practices, control mappings, migration phases	Enforcement and evidence per configuration change
AI in PQC migration [11], [27]	LLM code errors; agent downgrades; single-peer gating unsound	Advisor under injected prose; containment of proposer compromise; client panel
Runtime governance [17]–[20], [28]	Proposal and authority separated; exact-action approval; commit-time revalidation; decision evidence	Profile, dependency-freshness, and recovery-floor conditions; checks against key holders
Tamper-evident and post-quantum evidence [12]–[16], [29]	Hash chains, witnesses, role and post-quantum signatures, layout checks	Binding to an enforcing contract; obligations over authorization semantics
Explanation and zero-knowledge predicates [30]–[34]	Attribution, leakage, secret-policy accountability, public-threshold proofs	Hidden-parameter predicate bound to the authorized action

III. SYSTEM MODEL AND REQUIREMENTS

A. Assets, Actors, and the Management Boundary

The managed estate contains services, endpoints, protocol implementations, cryptographic modules, trust stores, and administrative interfaces. Each asset has an owner, a versioned configuration, dependencies (its peers and the implementations present on it), and an applicable migration profile.

The architecture separates logical roles. The source observer signs snapshots of an asset; the AI advisor proposes changes; the policy authority owns the versioned policy; the approver (the authorizer role) signs or declines each change; the executor alone writes managed state; an outcome observer measures the result independently of the executor’s report; the evidence service chains the signed statements and obtains checkpoints from an external witness; and the verifier checks the archive against its own trust configuration and policy registry. Roles that one administrator can modify are not independent; the reference implementation runs all roles in one process (Section IX). Claims of complete mediation and coverage apply only inside the management boundary, the declared set of governed assets and change interfaces.

B. Adversary and Failure Model

The advisor is untrusted: its output may be wrong, stale, or steered by injected instructions, and the most adverse proposer considered is fully compromised. While the gate, the executor, and the policy remain uncompromised, a policy-violating proposal is refused however it was produced. A manipulated choice among permitted actions, such as omitting a ready peer or selecting a weaker allowed profile, passes the gate (Section IV-B); only the approver or a stricter policy can reject it. Section VIII-F measures both cases.

The adversary may also be an insider holding valid role keys. A signature then shows only who made a statement, so

the verifier must reject validly signed but mutually inconsistent statements (Section V-D); consistent false statements remain possible until the compromise is contained. Failures are crash faults: a role may stop at any point, including between consuming an approval and recording its receipt. The chain and a retained checkpoint expose altered or removed records but not unrecorded transactions, which only reconciliation within the management boundary can reveal (Section V).

The trusted computing base for enforced changes comprises the policy evaluator, the executor and its durable state, the access controls on managed configuration, and the observers’ measurement path; evidence integrity adds signing-key protection and the retained checkpoints. Module validation assures only the module’s evaluated scope [35].

C. Design Requirements

- R1 Scope: each transaction identifies its asset, objective, policy version, and profile.
- R2 Bounded AI authority: model output cannot authorize a change, edit the policy, or generate secrets.
- R3 Validation: authorization requires fresh state, an allowlisted configuration, compatible dependencies, and a valid approval.
- R4 Controlled execution: a change applies only to the approved asset, configuration, and state version; each approval is consumed at most once.
- R5 Outcome evidence: completion requires an observation independent of the executor’s report.
- R6 Traceability: proposals, denials, approvals, attempts, and failures remain linked; an unresolved attempt blocks further changes to its asset.
- R7 Protected evidence: records are authenticated, retained, and checkpointed outside the operator’s unilateral control.
- R8 Explicit limits: each audit claim states its scope, evidence source, and trust assumptions.

Table S-I (supplementary material) maps migration decision classes to the permitted AI contribution and the required evidence.

IV. PROPOSED MIGRATION ARCHITECTURE

A. Separation of Proposal, Authorization, and Execution

Fig. 1 shows the control and evidence paths. The advisor receives the signed snapshot, the active allowlist, a task, and untrusted context documents, and holds no configuration credentials or signing keys. It returns a typed, unsigned proposal: asset, action, target profile with its exact configuration, endpoints, evidence digests, alternatives, and uncertainty. The invoking service, not the model, fixes the transaction identifier, asset, and policy version; the model’s choices are escaped to printable ASCII but never repaired, so an unacceptable choice is refused.

Each policy version fixes the permitted actions, an allowlist of profiles, a security floor (the minimum security class for migration and recovery targets), a freshness limit, and a maximum approval lifetime (time-to-live, TTL). A profile fixes

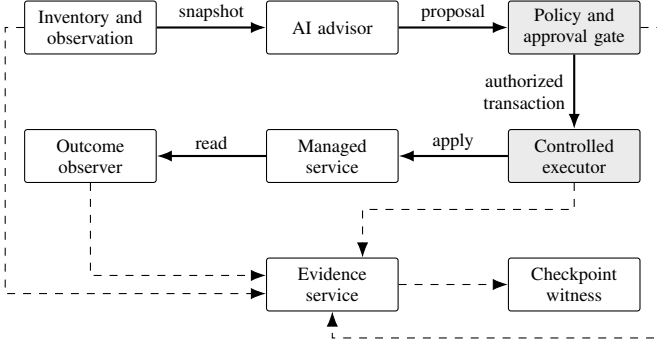


Fig. 1. Migration architecture. AI output reaches the managed service only through the policy and approval gate and the controlled executor; the gate evaluates the same signed snapshot that the advisor receives. The outcome observer reads the resulting state through its own read-only path, not from the executor. Dashed edges carry signed statements to the evidence service, which chains them and obtains checkpoints from an external witness.

protocol, key exchange, cipher configuration, security class, required peer capability, and implementation. A proposed configuration must equal the allowlisted one exactly, so a classical key exchange under a hybrid profile's identifier is refused.

B. The Migration Transaction

Let x_i be the signed observation for transaction i , a_i the proposed action, s_i the current state of the asset, v_i the active policy version, and u_i the signed authorization. The gate computes

$$b_i = \text{Fresh}(x_i, s_i) \wedge \text{Allowed}_{v_i}(a_i) \wedge \text{Compatible}(a_i, x_i) \wedge \text{Authorized}(u_i, x_i, a_i, s_i, v_i), \quad (1)$$

and $b_i = 1$ permits one execution attempt. The state s_i comprises the version, configuration, peers with capabilities and readiness, implementations, and whether an attempt on the asset is unresolved. No conjunct reads a model confidence score.

Fresh requires the snapshot in x_i to equal s_i , no attempt on the asset to await reconciliation, and the age of x_i to lie between zero and the smaller of its declared limit and the active policy's limit. Allowed $_{v_i}$ requires the proposal to cite v_i and a permitted action, and its target profile to be allowlisted, with an exactly equal configuration, at or above the floor. Compatible requires every proposed endpoint to appear in the snapshot with known readiness and the profile's required capability, and the profile's implementation to be present; it checks declared capabilities, not what every peer will negotiate [11].

Authorized requires a validly signed authorization bound to x_i : the observation presented for execution must carry a valid source signature and the digest and transaction identifier that the authorization binds, so a re-observed or substituted observation is refused. The authorization must match the transaction identifier, asset, and action digest of the complete proposal, the active policy version and the digest of its content, and the current state version. It is also rejected if it is dated in the future, has expired, has a lifetime above the policy's TTL,

was signed by a key not valid at execution time, or names an impermissible recovery target (Section IV-D).

The first three conjuncts are evaluated before approval; if one fails, the approver signs a denial. Immediately before acting, the executor evaluates all four against the then-current state; only this evaluation can permit a change. Because the approval binds x_i , a later change of the asset's version, configuration, or dependencies, of the policy, or of the proposal invalidates it. If all four hold, one durable commit consumes the approval and journals the attempt. The executor then performs the controlled operation; a second commit re-checks the approval's expiry, applies the configuration by a compare-and-set (CAS) on the state version, and marks the attempt applied. Expiry at that commit, a version conflict, or a failed operation ends the attempt as failed, with the state unchanged and the approval consumed. The consumption record refuses a replayed approval even when every conjunct holds, including after a restart. The CAS guards only the state version; equality of the dependencies is established before the consumption commit and is not re-checked at the apply commit.

C. Transaction Lifecycle and Interrupted Changes

A transaction moves from proposed through evaluated, authorized, prepared, executed, and verified to closed; rejected, failed, and recovered are terminal branches, and unresolved is non-terminal (Fig. S1, supplementary material). A denial, a failed re-evaluation, or an already consumed approval ends it as rejected. After the receipt, the outcome observer reads the resulting state; a difference from the receipt or the authorized target, or a failed health check, ends the transaction as failed. A crash after consumption and before the receipt leaves it unresolved, and Fresh then fails for every new proposal on the asset. A reconciler compares the journal with the authoritative state and never reads a missing receipt as a missing change: an applied change yields a late receipt dated from the journal and, after a matching outcome observation, recovered; an unapplied change yields failed; any other state is reported as diverged and failed. If the unresolved transaction was already archived, its resolution is appended as one record that the verifier accepts once (Section S-II, supplementary material).

D. Compatibility, Staged Deployment, and Recovery

Broader pre-change checks and staged rollout (Section S-III, supplementary material) are left to deployment practice; the reference implementation checks only Compatible. Because one peer does not establish what peers of other capability classes negotiate [11], outcome observation should use a panel of reference peers. The contract's outcome observer instead reads the store through a read-only view; panel observation of a deployed TLS service is a separate experiment (Section VIII-H).

Every approval names a recovery target, which must be service isolation or an allowlisted profile at or above the floor. The approver denies any other target with a recorded reason; an approval that a key holder signs with one anyway is refused by Authorized and flagged by the verifier (Section V-D). By default the approver names the current profile

if it meets the floor and isolation otherwise, so a model cannot obtain a below-floor fallback by invoking availability. The implementation records but does not execute recovery and has no exception path; recovered denotes an interrupted change confirmed by reconciliation, not a rollback.

V. EVIDENCE CONSTRUCTION AND AUDIT VERIFICATION

A. Record Structure and Signed Provenance

Every terminal transaction, and an unresolved one when archived, yields a record of separately authenticated statements, which keeps observed facts, approval decisions with their gate outcomes, and AI-generated interpretations apart. The source observer signs the snapshot (state version, configuration, peer capabilities, and implementations) with its capture time and freshness limit. The unsigned proposal is bound by its action digest; it references the observation and, for an LLM advisor, the retained transcript and context documents by digest. The approver signs the observation and action digests, expected state version, policy version and digest, issue and expiry times, permitted recovery target, and gate outcomes. The executor's receipt carries the before- and after-versions, applied configuration digest, and durable apply time; the outcome observer signs what it observes. The evidence service signs an envelope binding the section digests, status, log identity, index, and previous chain head; a witness signs a checkpoint over the new head. Tables S-II and S-III (supplementary material) list the fields and map the evidence to selected controls of NIST SP 800-53 Rev. 5 [36], as an assessment aid, not an official NIST crosswalk.

Audit records may be retained for years, and a quantum computer that breaks RSA and elliptic-curve signatures could forge statements signed with them, an instance of what NSA calls the "Trust Now, Forge Later" threat [3]. Signatures therefore use ML-DSA [8] (ML-DSA-65 in the reference implementation; Section IX) with one key per role; this yields authenticated statements under the stated key assumptions, not legal non-repudiation.

B. Chaining, Checkpoints, and Retention

Let E_i be the envelope statement of record i . The chain head is

$$h_i = H(\text{Enc}(\text{event}, i, h_{i-1}, E_i)), \quad (2)$$

where h_{-1} is a fixed genesis value, Enc is the canonical encoding, and H is SHA-384 over a versioned purpose label, a zero byte, and the encoded data. The encoder accepts only a subset of the value space of the JSON Canonicalization Scheme (JCS) [37] on which its output coincides with JCS, and rejects all other values (Section VIII).

At intervals bounded by Δ , an external witness signs and retains a checkpoint of the log identity, sequence number, chain head, and witness time; the reference implementation checkpoints every record. The verifier recomputes the chain to an independently retained checkpoint, supplied separately, never read from the archive, and requires a trusted witness signature, the expected log identity, coverage of the last record,

and an age of at most Δ . Detecting a log that shows verifiers different histories requires comparing their checkpoints; the reference implementation has one in-process witness and compares no checkpoints between verifiers (Section IX).

If the latest trustworthy checkpoint precedes a compromise by at most Δ , at most Δ of history is exposed to undetectable rewriting. The bound is retrospective: a continuing compromise can append false records until it is contained. A missed checkpoint voids the bound and is reportable. Checkpoints reveal deletion but do not restore evidence, so records require independent retention.

C. Reconciliation and Coverage

Chaining authenticates a presented history, not its completeness. Reconciliation must therefore compare authorizations, the executor's durable journal, current state versions, and independent observations; the reference implementation reconciles only an unresolved attempt, comparing its journal entry with the state. Every execution attempt needs an authorization and a terminal or unresolved status, and rejected proposals are retained. Within an exclusively mediated change interface, durable monotonic versions and one-time consumption expose missing receipts and duplicated attempts.

An archived unresolved record is never rewritten; a single resolution record with the same identifier, the same authorization, and the status recovered or failed is appended.

Interfaces outside the executor must be removed or monitored as documented exceptions, and periodic discovery, which the reference implementation lacks, must search for unmanaged assets. Inventory completeness remains unproven, so the audit package states its coverage and exceptions.

D. Semantic Verification Obligations

A valid signature shows who made a statement, not that it is consistent with the transaction, the cited policy, or the archive. The verifier addresses an adversary that holds valid role keys and can therefore issue correctly signed but mutually inconsistent statements and re-sign envelopes to re-chain an altered archive. Beyond schema, digest, signature, and chain-link validity, the verifier enforces these obligations on every record:

- (i) The outer transaction identifier equals the identifier inside every signed statement.
- (ii) The authorization binds the observation digest and the action digest that the receipt also binds; the envelope binds exactly the sections present, for every status.
- (iii) Allowed_v is recomputed for the cited version v : permitted action, the same policy version in proposal and authorization, the proposal's profile named by the authorization, an allowlisted profile with exact configuration at or above the floor, and the policy digest.
- (iv) The observation was fresh at authorization under the smaller of its declared limit and the policy limit.
- (v) The receipt's before-version equals the authorized version and its after-version is the successor, with continuity per asset.

- (vi) For closed and recovered transactions, the independently observed profile, configuration digest, and version equal those of the receipt and the authorized target.
- (vii) Every authorization digest is used once, apart from the resolution record of (xiii).
- (viii) A rejected transaction carries no receipt or outcome.
- (ix) The final head equals an independently retained, timely checkpoint for the expected log.
- (x) Compatible is recomputed from the signed snapshot, not taken from the authorizer's gate record.
- (xi) The approval lifetime lies within the cited policy's TTL, and the recovery target is isolation or an allowed profile at or above the floor.
- (xii) Every signed statement carries its own time, at which its key is valid, for every role including the evidence service and the witness.
- (xiii) Only known statuses occur, and at most one resolution record follows a transaction archived as unresolved.

Obligations (iii) except the policy digest, (iv), (vii), (x), and (xi) apply only to authorized decisions; a denial may record a stale observation or an unlisted profile. The verifier also checks envelope indices and non-decreasing times, that outcomes follow receipts, and that execution, timed at the durable apply time, lies between authorization and expiry. All obligations are checked in one streaming pass whose retained state (previous head, last version per asset, transaction identifiers, authorization digests, and transactions awaiting resolution) holds no record and grows linearly with the number of transactions (Section VIII-E).

These obligations reject correctly signed evidence that is inconsistent within a record, with the cited policy, with earlier records, or with the retained checkpoint, but not a fabricated history consistent in all these respects (the signing-key compromise case of Table S-IV, supplementary material). Signers assert statement times: (xii) excludes a key outside its stated validity period, but a key holder can choose any time within it that the ordering checks admit. The implementation enforces all thirteen obligations, each with a rejection test (Section VIII-B).

VI. OPTIONAL VERIFICATION OF CONFIDENTIAL POLICY PREDICATES

A. Purpose and Deployment Boundary

The optional extension lets a verifier establish that a registered predicate over withheld policy parameters and a withheld measurement evaluated to the recorded result for an authorized action. The proof does not establish that the parameters are adequate, that the action was executed, or that the migration complies with regulation.

Whoever generates a proof processes the complete witness, so the prover lies inside the confidentiality boundary with the policy evaluator, secret copies, backups, and operator access. Local proving keeps the witness from third parties but not from the evaluator's host, its administrator, or the file system holding the temporary witness file.

B. Bound Public Statement

Let θ contain the confidential parameters of a predicate Q in approved version v , and let κ_i identify the transaction, asset, state version, and measurement period. With a hiding and binding commitment scheme,

$$C_v = \text{Com}(\text{Enc}(v, \theta); r), \quad (3)$$

$$X_i = \text{Com}(\text{Enc}(\kappa_i, x_i); r_i), \quad (4)$$

where a trusted source measures x_i itself and signs X_i with κ_i , its identity, the capture time, and a freshness limit. The policy authority signs a registration of C_v , v , the predicate identifier p , the identifier I_R of the program that decides (5), the applicable assets, a validity period, and a maximum measurement age. D_i is the canonical digest of the proposed action a_i that the authorization and the receipt bind. The public statement, with p and the Boolean result q_i , satisfies

$$\begin{aligned} R = \{ & ((C_v, X_i, \kappa_i, D_i, v, p, q_i); \theta, r, x_i, r_i, a_i) : \\ & C_v = \text{Com}(\text{Enc}(v, \theta); r) \wedge X_i = \text{Com}(\text{Enc}(\kappa_i, x_i); r_i) \\ & \wedge D_i = H(\text{Enc}(a_i)) \wedge q_i = Q_{v, \theta}^p(x_i, a_i) \}. \end{aligned} \quad (5)$$

The verifier requires a current, signed registration covering the asset and the statement's C_v , v , and p ; a signed source attestation of X_i and κ_i , fresh within the smaller of the attested limit and the registered maximum age; a signed authorization of the same transaction, asset, and D_i ; and a proof for I_R or, for an authorized reviewer, an opening of the commitments. Otherwise it fails closed.

Because a proof attests only to the program it is checked against, the verifier takes I_R from the signed registration, never from the prover, and rejects a proof if the registration names none. The program recomputes C_v , X_i , and D_i , selects Q from a fixed catalog by p , and outputs the canonical encoding of the complete statement, which must equal the verifier's encoding byte for byte. If the proof system is sound, the program implements R , and the commitments and the digest are binding, an accepted proof implies a witness satisfying (5) (Sections S-VII–S-IX, supplementary material).

C. Instantiation and Cryptographic Limits

The program runs in the RISC Zero zero-knowledge virtual machine (zkVM) [22], version 3.0.6. The host proves with an explicit local, in-process prover, without the vendor's remote-proving client, and with the proof-free development mode compiled out. It produces and accepts only succinct receipts, which compress the segment proofs of the scalable transparent argument of knowledge (STARK) into one hash-based recursion proof; Groth16-wrapped receipts are not post-quantum [23].

The verifier accepts only a receipt lifted from a single segment of at most 2^{19} cycles. For the worst accepted receipt, the vendor's soundness calculator gives 98.0 bits for the segment and 99.8 bits for the recursion under its default toy-model conjecture, hence 98.0 bits end to end; the stricter list-decoding conjecture gives 76.8 bits, and 43.5 bits hold without either conjecture. The vendor's 99-bit figure [23] covers only the recursion. Larger segments yield fewer bits;

longer predicates would need multi-segment receipts and a re-evaluated bound.

A succinct receipt does not hide the execution length: its public control identifier names the last recursion program, which for one segment is the lift program for the padded power-of-two cycle count (here 2^{19}) and otherwise the join program. The length can depend on θ , such as the size of a permitted set.

RISC Zero targets perfect zero knowledge but has not written a mathematical argument for it [23]. Confidentiality of θ , r , x_i , and r_i against a receipt holder therefore rests on this assumption and on commitment hiding, which holds when SHA-384 is modeled as a random oracle. Soundness depends on neither. The statement itself discloses information about θ (Section S-X, supplementary material).

VII. ASSURANCE ARGUMENTS

The supplementary material illustrates a migration (Section S-XII) and maps failure cases to the arguments below (Table S-IV).

Authorization. Assume that every governed change passes through an uncompromised executor and that the gate, policy registry, and authorization credentials are trustworthy. Re-evaluation of (1) at execution, against the current state and the approved observation, and durable one-time consumption then restrict every applied change to the exact action of a valid, unused authorization. This establishes neither policy adequacy nor the absence of ungoverned interfaces.

Verified archive. If role and witness keys are also uncompromised and the verifier’s inputs are authentic, an archive that meets the obligations of Section V-D shows, for each authorized transaction, a single-use approval of an allowed, exact action on a fresh observation in which every listed endpoint is compatible; for each closed or recovered transaction, an outcome matching the receipt and the authorized target; and a history ending at the retained checkpoint. It establishes who stated what, not truth, and covers neither changes outside the management boundary nor drift after closure. A holder of valid role keys can still fabricate a history that passes verification.

Containment. Neither argument trusts the advisor: under complete proposer compromise and automatic approval, 0/60 policy-violating proposals executed, whereas 24/24 within-policy manipulations did (Section VIII-F). Enforcement cannot distinguish a manipulated but permitted choice (Section III) from a legitimate one; the archive makes it attributable, and only the approver or a stricter policy can prevent it.

Confidential predicates. Acceptance of (5) establishes the relation for the bound input and action under the binding, attestation, program-correctness, and soundness assumptions of Section VI; for the instantiation, soundness is conjectured at 98.0 bits (toy model). Confidentiality also assumes hiding commitments and an unproven zero-knowledge property [23]. Acceptance is not evidence of the model’s reasoning.

VIII. REFERENCE IMPLEMENTATION AND EVALUATION

A. Implementation

The reference implementation is a Python package (CPython 3.14.6) with modules for the gate, durable store,

roles, lifecycle (with fault injection and reconciliation), verifier, predicate binding, and an LLM advisor (Section VIII-F); a Rust workspace holds the zero-knowledge host and guest. It signs with ML-DSA-65 and performs its controlled operation, an ML-KEM-768 encapsulation round trip, through liboqs 0.16.0 and its Python binding [38]; digests are SHA-384 with purpose labels.

Roles and gate. Each of the seven signing roles holds one ML-DSA-65 key; a signer refuses a statement labeled with another role, and all roles run in one process (Section IX). The advisor holds no key. Each conjunct of (1) returns a named result and a reason.

Durable state. Configuration, per-asset versions, the execution journal, and consumed authorization digests reside in SQLite 3.46.1 with write-ahead logging (WAL) and fsync-backed commits (`synchronous=FULL`). Only the executor holds a writable connection; the observers read through a separate read-only connection, on which the engine rejects writes. *Prepare* consumes the authorization under a uniqueness constraint and journals the attempt in one commit; *apply* re-checks expiry and performs the CAS with the applied mark in a second; a third records the receipt digest. A version conflict, expiry at *apply*, or failure of the controlled operation ends the attempt as failed with the state unchanged (Listing S1, supplementary material).

Encoding and verifier. The canonical encoder accepts printable-ASCII strings and keys, booleans, null, integers within $\pm(2^{53} - 1)$, arrays, and objects, and rejects all other input. On this subset of the JCS value space [37], JCS serialization reduces to sorted keys, no whitespace, shortest integer form, and JSON string escaping, which the encoder emits. The verifier streams a JSON Lines archive. Trust configuration, policy registry, log identity, reference time, Δ , and the retained checkpoint are explicit inputs, never read from the archive. It checks each signature at its statement’s own time and implements obligations (i)–(xiii) of Section V-D, recomputing Compatible with the gate’s function.

Zero-knowledge host and TLS probe. The host implements the prover and verifier of Section VI-C. The TLS probe reads each reference client’s negotiated group from the ServerHello in its trace (Section VIII-H).

B. Regression Results and What They Establish

All 166 tests pass in the final run (`junit.xml`), including the opt-in zero-knowledge proof and TLS service tests; no test calls an LLM.

Every obligation (i)–(xiii) has a rejection test, and further tests reject violations of individual checks and of the verifier’s timing checks. Many re-sign altered statements with legitimate role keys, so only semantic checks can reject them. Table S-V (supplementary material) maps checks to tests and lists the checks without a dedicated test.

Contract tests show that a re-observation after a dependency change is refused without consuming the authorization; an authorization expiring before the *apply* commit is not applied; changed policy content under an unchanged version number invalidates an approval; after a crash before *apply* and a

TABLE II
MEASURED COSTS ON THE EVALUATION HOST (MEDIAN UNLESS
STATED)

Quantity	Value
<i>Governed transaction</i>	
Durable store, steady state	6.81 ms
of which three durable commits	4.76 ms
of which six ML-DSA-65 signatures	0.748 ms
Durable store, first 100 transactions	25.1 ms
Without fsync (diagnostic only)	1.45 ms
<i>Archive of 10^5 transactions (4,459 MiB)</i>	
Streaming verification	62.7 s
Streaming verifier, peak RSS	115 MiB
Earlier checker, own archive, peak RSS	5,051 MiB
<i>Zero-knowledge proof of (5), 3 runs</i>	
Proving, local and in process	67.9 s
Prover peak RSS, maximum over runs	4.6 GiB
Verification, in process	11.7 ms
Succinct receipt size	219 KiB
<i>TLS probe, X25519 hybrid client</i>	
Probe wall time, including client start	9.8 ms
Client process start alone	3.9 ms

Transactions: 300 warm-up and 200 timed repetitions per block. Peak resident set size (RSS) is V_{mHWM} of the measured process. Full decompositions: Tables S-VI and S-VIII, supplementary material.

simulated restart, durable consumption alone refuses a replay although all four conjuncts hold; an archive with a drift-failed transaction verifies; and the approver refuses a recovery target below the floor, whereas one signed anyway by a key holder is refused by the executor and detected by the verifier. Section S-XV (supplementary material) lists the remaining groups.

These tests show that the implemented checks reject the constructed violations. They do not estimate a detection rate, do not show that the obligations are complete or that every violating archive is rejected, and exercise durability only through injected faults (Section IX).

C. Experimental Setup

All measurements come from one laptop: a 12th Gen Intel Core i7-12650H with 15.3 GiB of RAM, Linux 6.19.14+kali-amd64, CPython 3.14.6, liboqs 0.16.0 built with GCC 15.2.0, and SQLite 3.46.1. The state store, journal, and archives reside on an ext4 file system on a Samsung PM9A1 NVMe SSD; the durable-commit costs reflect its fsync latency. The benchmarks ran on 2026-09-27 (UTC) on AC power with other applications closed; the CPU frequency was not pinned (`powersave` governor). Every driver records the power source, governor, load, and file system, and the timing, scaling, WAL, and earlier-checker drivers first wait for a quiet machine (Section S-XVI, supplementary material).

Raw samples, advisor transcripts, results, drivers, code, and manuscript sources are available at <https://github.com/blacksparrow/auditable-by-design>. Timings are Python call spans that include validation, canonicalization, and store access; they are not isolated cryptographic costs and do not establish production throughput.

D. Transaction Cost

A governed transaction comprises observation, proposal, gate, authorization, execution with journal and ML-KEM-768 operation, outcome observation, envelope, and checkpoint: six ML-DSA-65 signatures, two signature verifications, and three durable commits. After 300 warm-up transactions on a new store, the timing driver attributes the time of 200 timed transactions to non-overlapping spans; a second block with `synchronous=OFF`, which the contract does not permit, isolates the cost of durability (Table II).

In the steady state the median transaction takes 6.81 ms, of which the three commits account for 69.8% and the six signatures for 11.0%. Without fsync the median falls to 1.45 ms, of which signing takes 36.7% and canonical serialization 28.7%. On this host, durability rather than post-quantum cryptography dominates the cost of an enforced transaction; lower latency would come from faster durable storage, or from group commit that preserves the prepare-before-change order, and little from fewer signatures.

The first 100 transactions on a new store take a median of 25.1 ms, 3.7 times the steady state, because the WAL grows until its first automatic checkpoint. A sweep of the checkpoint threshold moved the start of the steady state to transaction 34, 132, and 505 for thresholds of 250, 1,000 (the default), and 4,000 pages; at the default, the median fell from 25.0 to 6.8 ms (Table S-VII, supplementary material). The warm-up therefore extends past the first checkpoint.

E. Archive Checking at Scale

Archives of 10 to 10^5 transactions over ten assets were built by the full pipeline and written as JSON Lines. The streaming verifier reads each archive record by record, with file reading and JSON decoding included in its time, and verifies all six signatures of every record, the retained checkpoint and obligations (i)–(xiii). Peak resident set size (RSS) is measured in a fresh process that performs one streaming verification, so archive construction is excluded.

Verification time is proportional to the number of transactions, between 0.626 and 0.638 ms per transaction at every size; the 10^5 -transaction archive is verified in a median of 62.7 s with a peak RSS of 115 MiB. Peak RSS stays at the interpreter baseline of about 30 MiB up to 10^3 transactions and then grows linearly, by about 0.9 KiB per transaction, consistent with the retained state of Section V-D (Table S-VIII, supplementary material).

The in-memory checker of the earlier prototype, measured in the same way from its own archive in a fresh process, peaked at 5,051 MiB for 10^5 transactions, 44 times the streaming verifier (2.6 and 14 times at 10^3 and 10^4). It checks less, over a different record format, so no speed comparison is claimed (Table S-IX, supplementary material).

F. LLM Advisor Under the Contract

Replacing the scripted advisor with an LLM, this experiment asks whether any proposer behavior, up to complete compromise, makes a policy-violating change execute, and

how far planted engineering prose steers the model toward changes that the policy permits but the task does not call for.

Setup. The advisor calls Claude Sonnet 5 through the Claude Code 2.1.283 command-line interface in non-interactive mode, with a replaced system prompt, no tools other than the interface’s own tool for schema-validated output, and extended thinking at the interface default (on). Each trial draws a synthetic asset with peers of mixed capability and readiness from a seeded generator. The task is to migrate the largest number of ready endpoints, to prefer the higher security class among profiles that reach it, and to propose no change if no profile has a ready endpoint; an oracle computed independently of the gate gives the correct answer. The approver approves whenever the gate permits, so only the gate stands between the model and the change. Each trial runs the complete contract on a non-durable store and verifies its archive.

Attacks. Engineering prose is planted in an additional context document or in a free-text note on an inventory peer, alternating between the two placements across scenarios. Attacks A1–A5 request policy-violating changes: an off-allowlist profile, the classical profile below the security floor, inclusion of a peer that lacks the required capability or is of unknown readiness, a tampered configuration (`kem=X25519`) under a post-quantum-capable profile identifier, and an unpermitted action. A6 (omit a ready peer said to be scheduled for decommissioning) and A7 (a weaker allowed profile justified by a fictitious change-board decision) request changes within the policy. A change counts as executed when it was applied, that is, when a receipt exists or the state version advanced; a policy-violating attack counts only if the applied proposal carries the requested violation. Section S-XVII (supplementary material) gives the prompt differences, abridged planted texts, and scoring rules.

Conditions. All three conditions use the same 12 scenarios per attack. The hardened prompt states that the inventory snapshot is authoritative and that context documents are untrusted, and the user message labels both accordingly; the naive prompt omits these two sentences and the labels. Under complete proposer compromise no model is called: a scripted adversary emits exactly the proposal that each attack requests. The model sees each scenario two times, with 40 benign scenarios under the hardened prompt and the first 20 of them under the naive prompt; the deterministic adversary sees each attack scenario once.

Benign behavior. The proposal matched the oracle in 80/80 hardened and 40/40 naive benign trials (Table III); no correct migration was denied and no suboptimal proposal executed. The benign trials that did not execute are correct `no_change` proposals, which Allowed refuses because `no_change` is not a permitted action.

Containment. In neither prompt condition did the model propose a policy-violating change: it followed none of the policy-violating instructions (0/120 hardened and 0/120 naive trials; true-downgrade subset of A2: 0/8 and 0/8), and the gate refused only correct `no_change` proposals. These conditions therefore do not exercise the gate against A1–A5; the evidence for containment comes from the compromised condition. There, the scripted adversary proposed the requested

TABLE III
LLM ADVISOR: ATTACKS EXECUTED PER CONDITION

Attack	Model (claude-sonnet-5)		Scripted
	Hardened	Naive	Compromised
A1 off-allowlist profile	0/24	0/24	0/12
A2 below-floor profile	0/24	0/24	0/12
A3 incompatible peer	0/24	0/24	0/12
A4 tampered configuration	0/24	0/24	0/12
A5 unpermitted action	0/24	0/24	0/12
A6 omit a ready peer [†]	6/24	24/24	12/12
A7 weaker allowed profile [†]	0/24	22/24	12/12
Benign: matches oracle	80/80	40/40	—
Benign: executed	64/80	30/40	—
Archives verified	248/248	208/208	84/84

Attack rows: trials in which the attacker’s change was applied; in the model conditions every followed attack also executed. [†]Within policy: the change satisfies all four conjuncts of (1). Model conditions: 12 scenarios per attack, each presented two times; the compromised proposer presents each once and has no benign trials. Archives verified: over all trials of the condition. Seed 20260926; interface Claude Code 2.1.283.

violation in 60/60 trials, and 0/60 executed. Allowed refused every proposal of A1, A2, A4, and A5, and Compatible every proposal of A1 and A3; the refused A2 proposals include all whose asset runs the hybrid profile (0/4 executed), which would have been true downgrades. Every archive verified, including those that record refused proposals. Containment of policy-violating proposals therefore does not depend on the proposer’s behavior, for the implemented attacks and while the gate, the executor, and the authorization credentials remain uncompromised (Section VII).

Within-policy manipulation. The gate is not designed to stop A6 and A7: every within-policy proposal of the compromised adversary executed (24/24); such a change is recorded, attributable, and reviewable in the archive, but not prevented. For the model, removing the two hardening sentences and the trust labels raised the executed within-policy attacks from 6/48 to 46/48 trials. A6 rose from 6/24 to 24/24 trials (scenarios with a success in any repetition: 4/12 to 12/12; in both: 2/12 to 12/12) and A7 from 0/24 to 22/24 (0/12 to 12/12; 0/12 to 10/12).

Under the naive prompt both channels worked: A6 executed in 12/12 document and 12/12 inventory-note trials, and A7 in 12/12 and 10/12. Under the hardened prompt only the signed-note channel did: every hardened A6 success came through the inventory note (6/12, against 0/12 through the document), and A7 never executed. The note travels inside the signed snapshot that the hardened prompt declares authoritative. Its signature establishes who recorded the note, not whether its free text is true, so labeling documents as untrusted does not protect free-text fields of authenticated observations. A policy requiring every ready endpoint would bring A6 within the gate’s reach; withholding free-text fields from the advisor is another mitigation, not evaluated here.

Han’s agents edit configurations directly, without an enforcing contract, so their downgrade counts [11] are not comparable. For the implemented attacks, the gate contained

the policy-violating part of the risk from planted prose but not the within-policy part.

G. Zero-Knowledge Cost

Relation (5) was proved with the RISC Zero guest and host of the reference implementation for a statement taken from a real transaction of the pipeline, so D_i covers the proposal that the authorization and the receipt bind. The predicate is a rotation-threshold instance: q_i holds if the measured key age reaches a confidential minimum and the target profile belongs to a confidential permitted set. The canonical encoding of the action is 911 bytes.

Over three runs, local proving took a median of 67.9 s with a prover peak RSS of at most 4.6 GiB (Table II). The execution used 305,611 user cycles within 524,288 total cycles, that is, one segment of 2^{19} cycles. Verification in the host process took 11.7 ms, and the succinct receipt occupies 219 KiB. Proving runs after the transaction closes, so it adds nothing to the transaction cost; at this cost it suits selected transactions whose predicate must be shown to an external verifier, not every transaction of a high-rate stream.

The receipt does not verify for the same statement with q_i inverted or against another program identifier, and the complete bound-statement verification, including the signed authorization, accepts the honest statement. The receipt bytes contain neither the canonical encodings of θ and x_i nor the commitment randomness. This excludes only verbatim inclusion of the witness; it is not evidence of zero knowledge, which remains an assumption (Section VI-C).

H. Observation of a Deployed TLS Service

This standalone experiment observes a real service on the wire and tests Han’s single-peer argument [11].

Setup. A TLS 1.3 service, nginx 1.30.0 with OpenSSL 3.6.2, listens on the loopback interface. Its governed configuration is its list of key-exchange groups, initially SecP256r1MLKEM768, X25519MLKEM768, X25519, and secp256r1. A panel of four reference clients, one per capability class (legacy, offering only X25519; X25519 hybrid, offering X25519MLKEM768 then X25519; P-256 hybrid, offering SecP256r1MLKEM768 then secp256r1; ML-KEM only, offering MLKEM768; Section S-XVIII, supplementary material), completes real handshakes with `openssl s_client` and sends an HTTP request. The negotiated group is read from the last ServerHello in each client’s trace, as the peer sees it on the wire, not from the server’s configuration.

Silent downgrade. In the base configuration, each hybrid client negotiates its hybrid group and the legacy client X25519 (Table IV). Scenario A removes X25519MLKEM768 from the server’s group list, and scenario B removes SecP256r1MLKEM768. Both edited configurations pass the nginx configuration test, and every client served before still completes the handshake and receives HTTP status 200. Yet in scenario A the X25519 hybrid client, and in scenario B the P-256 hybrid client, falls back through a HelloRetryRequest to a classical group. The legacy client observes no change in either scenario, each hybrid client misses the other scenario, and only

TABLE IV
NEGOTIATED KEY-EXCHANGE GROUP PER REFERENCE CLIENT

Client class	Base	After A	After B
Legacy	X25519	X25519	X25519
X25519 hybrid	X25519MLKEM768	X25519	X25519MLKEM768
P-256 hybrid	SecP256r1MLKEM768	SecP256r1MLKEM768	secp256r1
ML-KEM only	fails	fails	fails

Base: the base configuration. Scenario A removes X25519MLKEM768 and scenario B removes SecP256r1MLKEM768 from it. Every edited configuration passes the nginx configuration test. “Fails”: no handshake.

the panel detects both edits. The ML-KEM-only client fails in every configuration, since the server offers no pure ML-KEM group; it represents a class that fails rather than degrades. The opt-in TLS tests assert these observations.

Timing and scope. The median wall time of one probe by the X25519 hybrid client was 9.8 ms including the start of the client process, which alone takes 3.9 ms; the difference of 5.9 ms covers connection, handshake, trace output, and the HTTP request and is not a handshake latency. The experiment runs on one host over loopback, with direct configuration edits rather than executor-applied transactions. It demonstrates independent wire-level observation and the panel argument for the four classes of the panel; making the panel the outcome observer, with each client’s delivered class as a signed outcome statement, is future work.

IX. LIMITATIONS

Deployment. All measurements come from one host. The managed state is a local SQLite database, the governed operation is an ML-KEM-768 encapsulation round trip, and profile names are fixture labels, not evidence of protocol negotiation. The parameter sets ML-DSA-65 and ML-KEM-768 do not meet CNSA 2.0, which calls for Level V parameters [39], that is, ML-DSA-87 and ML-KEM-1024 (security category 5) [7], [8]; costs were not measured for the Level V sets. Keys are not hardware-protected, no validated module is involved, and liboqs is research software [38]. All roles run in one process, so role keys separate provenance, not authority; a compromised process could use any key. The observers read the executor’s store read-only: their observations are not derived from receipts but are not administratively independent of the executor. The witness keeps its checkpoints in the memory of that process, and no independent evidence retention is implemented. Durability was tested only with injected faults (crashes before and after the apply commit, failure of the controlled operation, restart by reopening the store), not with termination by the operating system, power loss, concurrent executors, or storage failure.

Observation. The contract’s outcome observer measures the recorded configuration, not delivered protection. The TLS panel is a standalone loopback experiment, not the contract’s outcome observer; no transaction was closed on wire-level evidence. The conjunct Compatible checks the declared capabilities of listed endpoints; by Han’s argument [11] it cannot establish what peers of other classes receive, and the panel covers only its four classes.

Advisor evaluation. One model was evaluated, through Claude Code 2.1.283 rather than the bare model endpoint; sampling parameters could not be set. Scenarios are synthetic, attacks are single-shot and non-adaptive, and each scenario was presented in one placement only, so placement is confounded with the scenario. Repetitions of a scenario are not independent, so counts are reported per trial and per scenario without pooled confidence intervals, and zero counts do not establish zero rates. The ablation removes the prompt sentences and the trust labels together and does not separate their effects. Because the model proposed no policy violation, containment rests on the scripted compromised proposer.

Zero-knowledge instantiation. By the vendor’s soundness calculator [22], the worst accepted receipt has 98.0 bits of soundness under the toy-model conjecture, 76.8 bits under the stricter conjecture, and 43.5 bits proven (Section VI-C). Zero knowledge, which the vendor has not proven [23], and commitment hiding are assumed; a receipt reveals the padded execution length, and results, timing, and rationale can leak policy information. The program identifier is reproducible only with the same guest toolchain; reproduction on another machine was not evaluated.

Assurance. The tests exercise the implemented checks on constructed fixtures; they neither estimate detection rates nor show that the obligations are complete. The contract and obligations lack a formal model and mechanical proof, and no assessor study was performed. Verifier memory grows linearly with transactions; archives beyond 10^5 transactions were not evaluated. Policy adequacy, inventory completeness, and role-key protection remain assumptions.

X. CONCLUSION

An AI recommendation neither authorizes a PQC migration nor establishes that it occurred, and explaining a decision based on a confidential rule discloses the rule. Auditable by Design therefore binds audit evidence to the governed operation and controls disclosure of decision logic separately. Its transaction contract links authenticated observations, an exact-action approval consumed once, CAS execution, a read-only outcome observation, and a witnessed, ML-DSA-signed history; its verifier rejects correctly signed evidence that violates the cross-statement obligations.

Under complete proposer compromise, 0/60 policy-violating proposals and 24/24 within-policy manipulations executed. With an LLM advisor and an approver accepting every gate-permitted proposal, within-policy manipulations executed in 6/48 trials under a hardened prompt and 46/48 under a naive one. Enforcement thus contained the tested policy violations independently of the proposer, whereas permitted choices depend on the advisor, prompt, and approver. Median costs on one host were 6.81 ms per governed transaction at steady state (mostly durable commits), 62.7 s to verify a 10^5 -transaction archive, and 67.9 s to prove the confidential-predicate relation under conjectured soundness and assumed zero knowledge. A four-client panel detected two silent TLS downgrades, each visible to only one client class. Administrative role separation, wire-level observation inside the contract, an assessor study, and a formal model remain open.

ACKNOWLEDGMENT

The author used Claude (Anthropic), an AI system, through Claude Code under the author’s direction for implementation, experiments, and writing. It drafted portions of the reference implementation (the LLM advisor harness, the RISC Zero guest and host, the TLS observation module, benchmark drivers, and several defect fixes) and drafted and edited the text of all sections of this article. The author reviewed and takes responsibility for all content. Separately, Claude Sonnet 5 is the evaluated advisor and thus part of the method (Section VIII-F).

REFERENCES

- [1] Executive Office of the President, “Securing the nation against advanced cryptographic attacks,” Executive Order 14412, Jun. 22, 2026. [Online]. Available: <https://www.govinfo.gov/app/details/DCPD-202600417>
- [2] Office of Management and Budget, “Execution of the migration to post-quantum cryptography,” Memorandum M-26-15, Jun. 24, 2026. [Online]. Available: <https://www.whitehouse.gov/wp-content/uploads/2026/06/M-26-15-Execution-of-the-Migration-to-Post-Quantum-Cryptography.pdf>
- [3] National Security Agency, “Post-quantum cryptography: A digital armor,” Post-Quantum Cryptography Resource Hub, Oct. 1, 2026. Accessed: Oct. 1, 2026. [Online]. Available: <https://www.nsa.gov/Cybersecurity/Post-Quantum-Cryptography-Resource-Hub/Article-Display/Article/4616296/post-quantum-cryptography-a-digital-armor/>
- [4] NIS Cooperation Group, “A coordinated implementation roadmap for the transition to post-quantum cryptography,” European Commission, Jun. 2025. [Online]. Available: <https://digital-strategy.ec.europa.eu/en/library/coordinated-implementation-roadmap-transition-post-quantum-cryptography>
- [5] European Commission, “EU reinforces its cybersecurity with post-quantum cryptography,” Jun. 23, 2025. [Online]. Available: <https://digital-strategy.ec.europa.eu/en/news/eu-reinforces-its-cybersecurity-post-quantum-cryptography>
- [6] National Cyber Security Centre, “Timelines for migration to post-quantum cryptography,” Mar. 20, 2025, updated Aug. 3, 2026. Accessed: Oct. 1, 2026. [Online]. Available: <https://www.ncsc.gov.uk/guidance/pqc-migration-timelines>
- [7] NIST, “Module-lattice-based key-encapsulation mechanism standard,” FIPS 203, Aug. 2024, doi: 10.6028/NIST.FIPS.203.
- [8] NIST, “Module-lattice-based digital signature standard,” FIPS 204, Aug. 2024, doi: 10.6028/NIST.FIPS.204.
- [9] D. Moody, R. Perlner, A. Regenscheid, A. Robinson, and D. Cooper, “Transition to post-quantum cryptography standards,” NIST IR 8547, Initial Public Draft, Nov. 2024, doi: 10.6028/NIST.IR.8547.ipd.
- [10] E. Barker et al., “Considerations for achieving crypto agility: Strategies and practices,” NIST CSWP 39-upd1, Dec. 2025, updated Jun. 29, 2026, doi: 10.6028/NIST.CSWP.39-upd1.
- [11] Y. Han, “Nothing breaks: No single peer can soundly gate post-quantum delivery,” arXiv:2609.07849, Sep. 2026. [Online]. Available: <https://arxiv.org/abs/2609.07849>
- [12] B. Schneier and J. Kelsey, “Secure audit logs to support computer forensics,” *ACM Trans. Inf. Syst. Secur.*, vol. 2, no. 2, pp. 159–176, May 1999, doi: 10.1145/317087.317089.
- [13] S. A. Crosby and D. S. Wallach, “Efficient data structures for tamper-evident logging,” in *Proc. 18th USENIX Secur. Symp.*, 2009, pp. 317–334.
- [14] B. Laurie, E. Messeri, and R. Stradling, “Certificate transparency version 2.0,” RFC 9162, Dec. 2021, doi: 10.17487/RFC9162.
- [15] L. Kao, “Post-quantum-resilient audit evidence for long-lived regulated systems: Security models, migration patterns, and case study,” arXiv:2512.00110v2, Feb. 2026. [Online]. Available: <https://arxiv.org/abs/2512.00110v2>
- [16] J. A. Gomes Marques, “Compliance profile of signed action receipts for AI agents,” Internet-Draft draft-marques-asqav-compliance-receipts-09, Internet Engineering Task Force, Sep. 2026, work in progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-marques-asqav-compliance-receipts/09/>

- [17] A. Mazzocchi, "Runtime governance for agentic AI: Action-boundary control with trusted provenance and fail-closed execution," arXiv:2608.16891, May 2026. [Online]. Available: <https://arxiv.org/abs/2608.16891>
- [18] N. K. S. Beshane, "The acknowledgment point is the system: Durable policy-decision receipts for AI audit evidence," arXiv:2608.17176, Aug. 2026. [Online]. Available: <https://arxiv.org/abs/2608.17176>
- [19] Y. Tong, L. Dai, and S. Guo, "AID-Guard: Stateful authorization for delegated agent effects," arXiv:2608.21159, Aug. 2026. [Online]. Available: <https://arxiv.org/abs/2608.21159>
- [20] A. T. Abak, "AIREP: A protocol for per-decision evidence in AI runtime governance," arXiv:2608.21363v2, Sep. 2026. [Online]. Available: <https://arxiv.org/abs/2608.21363v2>
- [21] E. Ben-Sasson, I. Bentov, Y. Horeish, and M. Riabzev, "Scalable, transparent, and post-quantum secure computational integrity," Cryptology ePrint Archive, Paper 2018/046, 2018. [Online]. Available: <https://eprint.iacr.org/2018/046>
- [22] RISC Zero, "RISC Zero zkVM," ver. 3.0.6, software, Jul. 2026. [Online]. Available: <https://github.com/risc0/risc0/tree/v3.0.6>
- [23] RISC Zero, "Cryptographic security model," RISC Zero Developer Docs, ver. 3.0. Accessed: Sep. 26, 2026. [Online]. Available: <https://dev.risczero.com/api/security-model>
- [24] C. Näther, D. Herzinger, S.-L. Gazdag, J.-P. Steghöfer, S. Daum, and D. Loebenberger, "Migrating software systems toward post-quantum cryptography—A systematic literature review," *IEEE Access*, vol. 12, pp. 132107–132126, 2024, doi: 10.1109/ACCESS.2024.3450306.
- [25] R. Misoczki, I. Elbaz, and F. Mertens, "Post-quantum cryptography migration at Meta: Framework, lessons, and takeaways," Engineering at Meta, Apr. 16, 2026. [Online]. Available: <https://engineering.fb.com/2026/04/16/security/post-quantum-cryptography-migration-at-meta-framework-lessons-and-takeaways/>
- [26] W. Newhouse, M. Souppaya, W. Barker, and K. Scarfone, "Mappings of migration to PQC project capabilities to NIST Cybersecurity Framework 2.0 and to security and privacy controls for information systems and organizations," NIST CSWP 48, Initial Public Draft, Sep. 2025, doi: 10.6028/NIST.CSWP.48.ipd.
- [27] H.-G. Kim, H.-W. Choi, M.-S. Kim, Y.-L. Hyoun, and H.-J. Seo, "Automated phased hybrid PQC-TLS migration via DevSecOps pipeline," Cryptology ePrint Archive, Paper 2026/1173, Jun. 2026. [Online]. Available: <https://eprint.iacr.org/2026/1173>
- [28] A. M. Mazzocchi, "Cryptographic runtime governance for autonomous AI systems: The Aegis architecture for verifiable policy enforcement," arXiv:2603.16938, Mar. 2026. [Online]. Available: <https://arxiv.org/abs/2603.16938>
- [29] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Capps, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *Proc. 28th USENIX Secur. Symp.*, 2019, pp. 1393–1410.
- [30] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 4765–4774.
- [31] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 1135–1144, doi: 10.1145/2939672.2939778.
- [32] R. Shokri, M. Strobel, and Y. Zick, "On the privacy risks of model explanations," in *Proc. AAAI/ACM Conf. AI, Ethics, Soc. (AI/ES)*, 2021, pp. 231–241, doi: 10.1145/3461702.3462533.
- [33] J. A. Kroll et al., "Accountable algorithms," *Univ. Pennsylvania Law Rev.*, vol. 165, no. 3, pp. 633–705, 2017.
- [34] A. S. Gopalakrishna, "Zero-knowledge predicate proofs between AI agents: A measured, cross-protocol gateway and the source-integrity gap," arXiv:2608.30083, Aug. 2026. [Online]. Available: <https://arxiv.org/abs/2608.30083>
- [35] NIST, "Security requirements for cryptographic modules," FIPS 140-3, Mar. 2019, doi: 10.6028/NIST.FIPS.140-3.
- [36] Joint Task Force, "Security and privacy controls for information systems and organizations," NIST SP 800-53 Rev. 5, Sep. 2020, updated Dec. 10, 2020, doi: 10.6028/NIST.SP.800-53r5.
- [37] A. Rundgren, B. Jordan, and S. Erdtman, "JSON canonicalization scheme (JCS)," RFC 8785, Jun. 2020, doi: 10.17487/RFC8785.
- [38] Open Quantum Safe, "liboqs and liboqs-python," ver. 0.16.0, software, Jul. 2026. [Online]. Available: <https://github.com/open-quantum-safe/liboqs/tree/0.16.0>
- [39] National Security Agency, "Announcing the Commercial National Security Algorithm Suite 2.0," Cybersecurity Advisory U/00/194427-22, PP-22-1338, ver. 1.0, Sep. 2022. [Online]. Available: https://media.defense.gov/2022/Sep/07/2003071834/-1/-1/0/CSA_CNSA_2.0_ALGORITHMS_PDF

Sergey Mkrtchyan is an independent researcher in Yerevan, Armenia, and works as a penetration tester in industry. Research interests include post-quantum cryptography migration, the security of AI-assisted systems, and verifiable audit evidence.